Cost: By decoupling the orchestration layer, you are also able to bring together context, models, and tasks in the most efficient and cost-effective way without sacrificing quality.

Compound: Bring these four together and you create your own continuous learning loop (i.e. hill climbing machine) that will allow your AI investments to compound the value of your firm.

In other words, a company should be able to use a model without giving up the knowledge that makes it unique. That is the reverse information paradox we need to confront.

FRED TALKS

13th October 2026

CONTENTS

The Self-Help Trap: What 20+ Years of “Optimizing” Has Taught Me

TIM FERRISS · 3114 WORDS

Ownership

THORSTEN BALL · 1122 WORDS

Giving and taking credit in big tech companies

SEANGOEDECKE.COM RSS FEED · 1543 WORDS

The Reverse Information Paradox

SN SCRATCHPAD · 819 WORDS

The Self-Help Trap: What 20+ Years of “Optimizing” Has Taught Me

TIM FERRISS · 08 MAR 2026 · SOURCE

One danger of modern self-help.

> **“We cannot reason ourselves out of our basic irrationality. All we can do is to learn the art of being irrational in a reasonable way.”**
> — Aldous Huxley, *Island*

It was cold out, but none of us were cold.

I sat with five men in the mountains of Montana. As the sun set, the fire in the center cast dancing light on our faces. Reclined against fallen trees in a tight circle, we ate mushrooms and fish we’d found under trees and along streams. The whole crew burst into laughter yet again, and one of the guides passed around a fresh batch of pine needle tea.

Bathed in warmth, I took off a layer and glanced skyward through an opening in the trees. The stars shone like crystals on black velvet, and the show—the biggest meteor shower of the year—was starting.

In that moment, there was nothing to do. Nothing to improve. Nothing to fix.

It was perfect.

***

The older I get, the more I think that self-help can be a trap. Sometimes the cure is worse than the disease. I say this after ~20 years of writing self-help and a lifetime of consuming it.

Spend enough time in the world of “improvement,” and you’ll notice something strange: The people most obsessed with self-help are often the least helped by it. Behind the smiles and motivational quotes, behind closed doors and after a drink or two, the truth is that they’re not able to outsmart their worries.

On one hand, perhaps this unhappiness is precisely what lands one in self-development in the first place, right? I long assumed this about myself, and it’s partially true.

On the other hand, what if self-help *itself* is actually creating or amplifying unhappiness?

While the great innovation that comes from model providers having fair use rights to train models on public data is needed, I find it ironic that the status quo is to then turn around and impose restrictive terms on distillation, and to reserve the right to learn from customer usage and interaction data. If learning flows in only one direction, economic value converges toward the owners of the learning infrastructure rather than the creators of the knowledge itself. Therefore, it’s imperative that we distribute the learning infrastructure to every firm so that they can control their own learning loop.

As Alex Karp put it: “What the technical customers want is control over their compute, their models, their data stack, and their alpha. They want to know they own the means of production, and it’s not being transferred to someone else.” The current regime does precisely the transfer Karp and companies fear.

That is why enterprises need a real trust boundary for their human capital and token capital to compound. It is where an organization’s data, traces, evals, adapted weights, and memory accumulate and improve together. And it is a hard boundary across which nothing crosses, not even the intelligence exhaust, without consent. Enterprises will demand the rights to use model outputs to fine tune and/or train their own models. I think of this as every firm’s right to align models to their enterprise accountability obligations.

In the cloud era, enterprises accumulated data. In the AI era, they accumulate learning. The trust boundary must evolve accordingly, from protecting information to protecting the mechanisms through which organizations learn, adapt, and compound intelligence. There are a few things every enterprise must do to ensure this:

Control: Create your private evals, because evals define what “good” looks like inside the organization. Also, retain ownership of your organization’s memory, traces, feedbacks, decisions, and institutional context, and ability to use outputs of models from your own tasks and queries.

Capability: Build your own proprietary learning environments within the tenant boundary to train or tune models, where models learn against real workflows without exposing the company’s knowledge.

Choice: Ensure the orchestration layer is decoupled from any single model. Ask yourself: If any one model you are using is taken away, do you still have the ability to operate and optimize for your evals using other models? Does your company “veteran” capability remain with you even if a given “generalist” model is taken away?

The Reverse Information Paradox

SN SCRATCHPAD · 05 AUG 2026 · SOURCE

Notes on advances in technology and real-world impact

In the age of intelligence, how should firms protect their core IP?

Nobel Prize winning economist Kenneth Arrow famously described a paradox in the market for information. “Its value for the purchaser is not known until he has the information, but then he has in effect acquired it without cost.” In Arrow’s “Information Paradox,” the seller risks giving away knowledge in order to sell it.

AI creates the reverse problem. In the AI age, the buyer risks giving away knowledge, just in order to use what they bought.

You essentially pay for intelligence twice, once with money, and again with something even more valuable: the proprietary knowledge you must reveal to make that intelligence useful. The better you want the model to perform, the more of that knowledge you have to feed it!

Over time, the information asymmetry becomes increasingly skewed. The seller learns more and more about you as you use what you purchased, while you learn very little about what the seller is learning in return.

That is what I think of as the Reverse Information Paradox.

Patents solve one aspect of Arrow’s paradox. They let an inventor disclose an idea without simply giving it away. The Reverse Information Paradox needs its own equivalent.

This requires more than data protection. Models learn from “exhaust,” the prompts people write, the tools agents use, and especially the corrections people make when the model is wrong. Every correction is distilled into institutional know-how. It’s the kind of knowledge a competitor could never buy, and the kind that leaks almost imperceptibly: trace by trace, correction by correction, eval by eval.

In consuming intelligence, you are creating intelligence. And what you create should belong to you. This is your particular intelligence, in Hayek’s sense: the knowledge of time, place, and circumstance that no one else can hold. It knows what you think, what you value, and how you measure success.

Modern self-help contains an in-built flaw:

To continually improve yourself, you must continually locate the ways you are broken.

Fortunately, there are a few perspective shifts that make all the difference. It took me embarrassingly long to figure them out.

To get started, let’s take a fresh look at an old concept.

MASLOW’S HIERARCHY OF NEEDS?

> **“I suppose it is tempting, if the only tool you have is a hammer, to treat everything as if it were a nail.”**
> — Abraham Maslow

Maslow’s Hierarchy of Needs has captured the minds of hundreds of millions. It offers simplicity in a terrifyingly complex world.

Abraham Maslow’s “A Theory of Human Motivation” (1943) contains five levels, which are typically presented like the below pyramid. This one is pulled from the Wikipedia entry on the subject:

We’ve all seen it. Clear as day, you can see the goal post at the top: self-actualization.

LFG! It’s time to journal and 80/20 myself! Pass me a shaman and some modafinil.

That’s the mission. That’s the point.

Right?

But hold on. A critical footnote got lost in the shuffle. In his later writings, especially notes compiled in *The Farther Reaches of Human Nature* (1971), Maslow added a *sixth* level above self-actualization:

Self-transcendence.

That update never quite made it out of the crib. The consultants are to blame, but that comes later.

Self-transcendence means going beyond the self — seeking connection with something greater, such as service to others, nature, art, or the divine. Why is it important? Well, for one thing, as Tony Robbins put it at an event long ago: "'I, I, I, me, me, me' gets to be a really fucking boring song."

But it's not just a boring song; it's dangerous to your health.

DON'T BE A SOMO

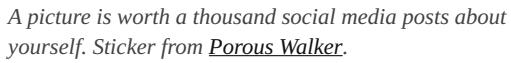
> **"The man who renounces himself, comes to himself."**
> — Ralph Waldo Emerson

Self-help is dangerous precisely because it easily becomes self-fixation.

A focus on improving the self usually first requires finding problems with the self. This is quite the pickle. In a society that rewards problem-solving, you can end up hallucinating or exaggerating unease in order to fix it. This leaves you always in the red, always one step behind. Imagine a dog chasing its tail that has committed to being unhappy until it catches the tail… but it's always just a few inches short. Still, it whirls around and around, "doing the work." Perfection always recedes by one more book, one more seminar, one more habit tracker.

Put in more colorful terms, misdirected self-help turns you into a self-obsessed masturbatory ouroboros (SOMO).

To remind me of the SOMO risk, I have this sticker on my laptop:



A picture is worth a thousand social media posts about yourself. Sticker from Porous Walker.

Now, to be clear, I still love self-help. Ain't no way Timmy can give up the sauce. There's a place for it.

From The Bible to Seneca, and from Ben Franklin to Stephen Covey and far beyond, there's a lot of valuable advice worth taking. I used to mainline it all – no time to waste! – and jump straight into action. This did some good, but there was a lot of collateral damage.

Why?

Because there are at least three "tectonic plates of self-help" that I couldn't see for decades, and they dictate how much net-positive or net-negative comes from all the striving. Before you sprint, you want to calibrate your direction.

the brand-new thing that they don't understand and are not responsible for. If instead that network had been involved in your project — if they'd been in a position to share the credit — they'd be less incentivized to blame it.

Of course, engineers are (mostly) not scheming viziers who make purely self-interested decisions. When asked who to blame, they usually make a good-faith effort to answer honestly. But in an area where there's no single clear right answer, it's human nature to be at least a little bit guided by your incentives. Nobody likes to think they're responsible for a group failure.

Conclusion

Credit and blame are the currencies of tech companies (and often directly translate to the actual amount of currency you get to take home). For technical roles, managers assign credit and blame based on lots of quiet conversations with their trusted engineers. This can be a rude awakening for very junior engineers who are used to having their work assessed by an expert grader (or less junior engineers who haven't yet shaken that mindset completely).

Don't expect to get credit simply by putting your head down and doing good work. You have to find some way to tell people what you're doing and why it's important: internal blog posts, mentioning it in 1:1s with your manager, or anything else you can think of. But don't take self-promotion too far. It's a bad idea to try and hoard all the credit for your projects, for two reasons.

First, sharing credit with other people gives them a reason to talk positively about your project. Credit is not a zero-sum game: if you do it right, you can get other people to build up your credit for you. Second, hoarding credit sets yourself up as a lightning rod for blame. Projects where the credit is concentrated in one or two people are automatically[2] blamed for complex problems, because nobody is incentivized to defend them.

1. This is a classic example of an illegible-but-essential part of a software company. I wrote about this general phenomenon in *Seeing like a software company*.

   ↩

2. Of course, if you do really screw up, you'll be blamed no matter what. I'm talking here about complex failures where it's non-trivial to attribute blame to a single source.

   ↩

vulnerable to other people coming in to claim credit. You can tell they're jealously guarding whatever credit they've managed to accumulate. The lesson this kind of engineer hasn't learned is that **you can often accumulate credit best by giving it away**.

To see why, consider how credit flows *up* inside a tech company. I wrote above that your manager can't assess the quality of your technical work on their own, but instead has to rely on other engineers they trust. They'll quietly ask those engineers "hey, was this project really that impressive?". In fact, often there are multiple layers of this at play[1]. In big companies, line managers usually don't decide who gets promoted or who gets a raise: they make recommendations to their manager, who has their own network of trusted engineers (confusingly, sometimes these networks overlap). The point is that **there is a large group of people behind the scenes who will quietly and informally judge the value of your work**.

Succeeding at a tech company is largely about finding ways to get these people on your side. The easiest way is to share your credit with them — and since you don't know who exactly is in this group, you should be sharing your credit freely. When you get feedback from other engineers, publicly thank them and mention them in your internal posts about the project. Find opportunities to ask for small favors, so you have an excuse to give other people credit. As best you can, make your individual projects at least partially *group* projects.

Sharing credit with others gives them a reason to support you. A shared project you've worked on reflects well on everybody: on you, for working well with others, on the people you've worked with, for the same reason, and for your manager, for fostering such a great environment of cooperation. Lots of people have good reason to talk that project up, because it's partly their project too. On the other hand, a project you've jealously kept to yourself reflects well on nobody: you come across as antisocial and your peers come across as unhelpful.

Blame

Blame operates by the same rules as credit. When something goes badly wrong, managers will ask their networks "hey, who screwed up here?" The answer to this question is never simple. Even on a purely technical level, failures always involve an interaction between multiple complex systems, any one of which could conceivably have been built so as to avoid the failure. In other words, **competent engineers can assign blame pretty much wherever they want**.

Because of this, it's risky to have a project for which you're clearly the only one getting credit. When something goes wrong, the network of people who will assign blame will likely be implicated in every part of the system but yours. They will be incentivized to attribute fault to

THE THREE TECTONIC PLATES OF SELF-HELP

> **"As to methods there may be a million and then some, but principles are few. The man who grasps principles can successfully select his own methods. The man who tries methods, ignoring principles, is sure to have trouble."**
> — Harrington Emerson

In the last few years, my life has become much more of a joy than a grind, and that's because I've focused on three tectonic plates.

Let's take a close look at each.

1. Intention

Individual or Social?

Americans, in particular, worship at the altar of the rugged individualist. There are clear upsides to this. But steeped in a culture – offline and especially online – that puts the self on a pedestal, we can take self-improvement to be an end unto itself: a better self.

But is it an end unto itself? Does it automatically produce good things? I now have my doubts.

Here's one analogy I've drawn for myself.

Let's pretend that life is the game of soccer. You can work on the mechanics of soccer by yourself. You can always get better at dribbling, shooting, and running drills as a solo practitioner. You can read dozens of books, study tape, and earn a PhD in the physics of ball flight. You can post videos of stunning shots on YouTube and get showered by emojis.

But none of this is actually playing the game of soccer.

You can spend your whole life preparing for, instead of playing, the game of life.

But why would anyone, including yours truly, succumb to this?

Subconsciously, it spares you from the messiest but most rewarding game of all: human interaction. Perhaps people hurt or traumatized you long ago. You might also justify the endless polishing, as I did, with some version of "Once I've perfected myself, then I'll be ready for relationships." But here's the rub: that practice is exactly endless. You can always get better at dribbling and penalty kicks.

Digging further, focusing on improving the self is often in service of trying to control the world, especially if things were unpredictable or unstable when growing up. Banish emotion, live by spreadsheets, and all can be well. All can be *controlled*, or so the illusion goes. But as soon as you're interacting with – let alone depending on – other people, control as a construct goes out the window. And so we consciously or subconsciously avoid the messiness. This is also one of the reasons why a lot of optimizing achiever folks have a hard time in intimate relationships.

So how do I think about "self-help" now, having realized all of the above?

It is refreshingly simple: the goal is to build and improve my relationships. The sooner you get on the real field with real players, the sooner you can get to playing soccer and engaging with life. No more auto-fellating, even with the best of intentions. We've evolved over millions of years to be deeply social creatures, and the more you dodge that IN REAL PHYSICAL LIFE, the more you will suffer. This is why solitary confinement in prisons is often considered cruel and unusual punishment… and yet we do it to ourselves all the time.

There are a few questions that help corral this tectonic plate of *intention*:

- How does any given "self-help" help me in my relationships, and how can I apply it with other people *today* or *this week*?

- How can I take the ship out of the harbor and test it where it counts?

2. Audience

Do you have an audience for your self-development? If so, be careful.

Nary a minute can be spent on social media without bumping into a CAPS-rich "HOW X CHANGED MY LIFE" or a photo carousel of an ayahuasca retreat. If only Costa Rica got a dime for every bikini-clad healer under a waterfall!

Welcome to the theater of performative self-help. I won't belabor this, as we've all seen it, but I suggest reading about the insidious creep of audience capture here, and don't forge ahead in the fame game before reading 11 reasons not to become famous. It's hard to put the genie back in the bottle, so you should know what that genie will do to your life.

But the truth is that most of us aren't extreme examples of this. But even minor tendencies in this direction can do extreme damage over time.

Many big tech companies encourage this attitude, because it helps them recruit smart graduates. They fashion their workplaces to look and feel like a university, even calling the physical space "campuses". But it's still work, not school. If you treat it like school, you are going to have a bad time.

Taking credit

The first lesson many new engineers learn is that **you have to take credit for your work**. If you silently jump in to help a struggling project and get it back on track, there's no guarantee of reward. Credit will naturally flow to the project lead, not you. In fact, if this project is outside of your direct team, it's likely you will be *punished* for it: to your manager, it will look like you're simply doing nothing at all.

Even when your manager is watching your work, credit is largely uncorrelated with how well you did. That's because, unlike at school, **you are the subject-matter expert on your own work**. Software systems are so complicated that only the people who work on them can hope to understand them, and even that understanding is always imperfect. If even experts can't reliably estimate the difficulty of changes, how is your manager supposed to assess your technical performance? The answer is they aren't. They're simply not qualified to assess it.

Instead, smart managers will find engineers on your team they trust and ask them how you're doing. On small teams that have worked on a single codebase for a long time, this works okay, because everyone's familiar enough to judge everyone else's work. On large teams with a high rate of codebase churn, it goes badly, since they're just guessing. On teams with a nasty, cutthroat culture, it sometimes goes *very* badly, since this is a good opportunity to actively sabotage the engineers who might threaten you.

Experienced engineers know how to **take the credit themselves**. When they do something good, they tell their manager about it. They write internal posts explaining why it was technically difficult and how they solved it (the audience for these is partially those trusted engineers, and partially the managers who will see a long technical post and think "wow!" without reading it). They actively build trust with their management chain. Worrying about this stuff is the beginning of playing politics.

Giving credit

There's a kind of engineer who's learned how to take credit but hasn't learned any other lessons yet. They're proactive about telling people what they've done, and they always maintain a "brag doc". In particular, they love to talk about the parts they did *by themselves*, since those are least

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, … — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.



You know what *you* should own? A subscription to this newsletter:

Giving and taking credit in big tech companies

SEANGOEDECKE.COM RSS FEED · 02 AUG 2026 · SOURCE

Engineers often complain that visibility should be their manager's job. In other words, they think engineers should be able to focus on the code, while their manager figures out who's doing well and rewards them.

This attitude is an extension of the "school fantasy": the idea that your workplace should operate by the same rules as your school or university. After all, you didn't have to worry about "visibility" during your education. You simply did the assignments and tests you were given, and if you did well you were rewarded with a good grade.

Below are a few questions that I've found helpful for nudging this particular tectonic plate in the right direction:

- If you couldn't tell a soul about "the work" you're doing, would you still do it? If not, you're not developing yourself; you're curating yourself.
- How has sharing your personal development created tradeoffs?
- If you *had to* take down 20% of your most popular posts, which would you take down and why?
- Are you describing strong catalysts (psychedelics, The Hoffman Process, you name it) instead of doing the post-session integration that makes them truly valuable?
- Have you become more robust or more fragile by offering your inner workings up to public vote?
- Has your social presence made you more or less of the person you want to be? How would the you of three or five years ago feel about your last year of posts? What about the you of 10 years from now?

3. Assumption

What are the fundamental assumptions behind your doing "the work"?

Let's begin with a Buddhist parable that I first heard from the incredible Jack Kornfield.

> *The old Master points to a big boulder and asks a disciple, "See that large rock over there?"*
>
> *"Yes", says the disciple.*
>
> *"Do you think it's heavy?" continues the Master.*
>
> *"Yes, it's very heavy!" replies the student.*
>
> *"Only if you pick it up," smiles the Master.*

Once again, the fundamental assumption behind self-help is often this: Something is not OK. Something is wrong. Something is not enough. Something needs fixing. If I can't find it, I'll create it.

We've established this. But there is a follow-on assumption that matters a lot.

If I fix the things that aren't OK, all will be well. If I improve myself enough, if I only work hard enough, I can finally eliminate my suffering.

I hate to inform you, but this doesn't work. I'm also thrilled to inform you that this doesn't work. You can stop picking up a lot of boulders.

There is one book that most opened my eyes to this reframe – *Already Free: Buddhism Meets Psychotherapy on the Path of Liberation* by Bruce Tift. It offers a terrifying but ultimately liberating realization: there is no perfect escape from suffering. It doesn't exist. But there is a way to find your long-sought unclenching, and it lies in cultivating your skill of acceptance as much as that of improvement.

Now, I can hear the chorus: *Has Tim gone soft? Given up the good fight? Is he telling everyone to chill after he himself red-lined and got the spoils? How convenient! And…*

Hold on a second. I'm telling you – intelligent acceptance is high-leverage. It's probably one of the highest forms of leverage. This is an approach that helps preserve your energy for where it really matters. My early forays into Stoicism and Seneca The Younger helped set the conditions for my biggest wins from 2004-2010. Still, I only learned a small fraction of what I needed.

So how do you cultivate your skill of acceptance without becoming complacent?

This is a big question and what I love about Bruce's book. Compared to a strictly Western or purely Eastern book, he blends them and offers a surgical guide to using *both* action and acceptance. You don't have to be a bull in a china shop or a cow in the rain; there is a middle path. That middle path is where all the gold is buried.

If the only tool you have is "self-improvement," you'll become a hammer looking for nails in a world that is 50% screws. I tried it. It can create the veneer of success, but it will leave your inner world in turmoil.

Suffice to say, the dual dance is the most joyful. Upgrade your toolkit with that in mind. Read Bruce's book. If it doesn't click, try *Radical Acceptance: Embracing Your Life with the Heart of a Buddha* by Tara Brach, which had a large impact on my life a decade before I found Bruce's book. In a sense, the writing of Seneca prepared me for Tara, which then prepared me for Bruce. So grab them all and thank me later.

If you want serenity, you need to be able to put the Serenity Prayer into practice. Seriously, I read it all the time.

- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say "here's what I built, under these constraints, with these tradeoffs?"

- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?

- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?

- If you think your colleagues needs to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.

- Do customers need to know? Who reported the bug? Who's blocked? Let them know.

- Does the world need to know? Announce that it's out.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.



"How does this apply juniors? You can't expect them to really do all of that?"

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.



Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase “ownership” or allude to it, but haven’t explained what “ownership” means to me in a while.

So, ownership.

If I ask you “can you own this?” or “can you take care of this?” or “are you on it?” — what I’m doing is I’m asking you to *own* it, to own the solution of a problem from end to end. From “we have a problem” to “we don’t have to think about it again.”

That means, when you say that you’re owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we’re actually trying to solve here. Maybe you think “the problem is that we need to migrate from using X to using Y”, but that’s not a problem, that’s a solution. The problem is likely something like “performance is bad”, “it’s not stable”, “it fails for customer x”. Maybe there’s other possible solutions to that? Think about those. What are the tradeoffs? What’s the best solution to go with considering these tradeoffs?

- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?

- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?

- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven’t confirmed yet?

- Think about how you’d test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?

- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!

MASLOW’S HAMBURGER OF NEEDS?

> **“The more one forgets himself—by giving himself to a cause to serve or another person to love—the more human he is and the more he actualizes himself. What is called ‘self-actualization’ is not an attainable aim at all, for the simple reason that the more one would strive for it, the more he would miss it. In other words, self-actualization is possible only as a side-effect of self-transcendence.”**
> — Viktor E. Frankl

How can we easily keep ourselves on the right track?

As I remind myself these days: **It’s the relationships, stupid.**

For a nice simple visual, let’s revise Maslow’s pyramid with all of this in mind. This is easy, as Maslow never drew his model as a rigid pyramid!

He described “classes” of needs that were unfixed, overlapping, and that could reverse in order. And believe it or not, self-actualization was only ever for the “self-actualizing minority.” In the 1960s, his work was co-opted by consultants and corporate trainers who needed a progression to sell. True story.

Given all this, and after decades of trial and error, here’s where I’ve landed:

Maslow’s Hamburger of Needs.

Ahhh… what? Not to worry. It’s the same good ol’ Maslow ingredients, but I think of it as a hamburger:

For our purposes, the meat, the whole point of the hamburger, is that middle layer: relationships. That is the center of life. The heartbeat.

As luck would have it, when you improve the heartbeat, it also feeds everything else.

You’ll notice that the meat contains Abe’s most-important addendum—the sixth level of self-transcendence. Focusing on things bigger than yourself is a critical piece of the ultimate puzzle. Faith, nature, family, meditation, causes that outlive you, etc. – take your pick. But be careful. If you do it to inflate the ego or impress others, it’s self-obsession again, not self-transcendence. If you need credit, it doesn’t count.

Of course, it should go without saying but the top and bottom layers matter a lot. A hamburger is a giant mess without the bun. Friends will get sick of you crashing on their couch and eating their food.

But the bread and dressing layers exist to serve the middle. That's the payload. Everything is in service of the payload. And the payload circulates benefits back to the edges, and then the cycle repeats. Even if you think this is oversimplified claptrap, temporarily assuming it's true will help you.

What if nearly everything you focused on — calendar, habits, goals — aimed to improve your relational life somehow? What if you took this as a challenge for even a week? Your lens on the world changes dramatically.

You say yes differently.
You say no more clearly.
Your to-do list for life slowly transforms.

What if all that you focused on, all that you do, had to improve that middle layer in some fashion?

It's a damn hard question if you've been on the *self*-help train for a while. I get it.

So let's try something easier: **What if it only changed how you approach your to-do list? Try hamburger-first each day for 1-2 weeks and tell me what happens. Add and do the things that improve your relational life FIRST. Nothing on the list? Create something. It could be as simple as cooking dinner for your spouse, complimenting at least three people a day for a week, or introducing yourself to the barista you see every morning. Getting started is how you get grooving.**

ARE YOU DOING SELF-HELP, OR IS SELF-HELP DOING YOU?

> **For friendship makes prosperity more shining and lessens adversity by dividing and sharing it.**
> — Marcus Tullius Cicero

In his *Moral Letters to Lucilius*, Seneca the Younger famously wrote that "These individuals [who put money at the center of life] have riches just as we say that we 'have a fever,' when really the fever has us."

What if self-help is similar?

Obsessing over the self never provides peace. It cannot make you whole, as you aren't the whole. Becoming whole starts by putting down the rock you didn't even know you were carrying.

Because at the end of the day—and at the end of a Montana night—the point was never yourself.

It was never the pyramid.

It was never the optimization.

It was the people around the fire.

Share this:

- Share on X (Opens in new window) X
- Share on Facebook (Opens in new window) Facebook
- Share on LinkedIn (Opens in new window) LinkedIn
- Email a link to a friend (Opens in new window) Email
- More

Ownership

THORSTEN BALL · 08 JUL 2026 · SOURCE

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.